

File permissions in Linux

Project description

The purpose of this activity is to manage file and directory permissions in Linux using commands such as `ls -la` and `chmod`. I examine the existing permissions in the `projects` directory, identify permissions that do not follow the organization's security requirements, and modify them appropriately. This demonstrates how Linux authorization can be used to protect files and restrict access to sensitive directories.

Check file and directory details

`ls -la`

I used the `ls -la` command to display detailed information about all files and directories in the current directory, including hidden files. The `-l` option displays the long-format information, including ownership and permissions, while the `-a` option displays all files, including hidden files.

Example permission string:

```
-rw-rw-rw-
```

This indicates that `project_k.txt` is a regular file. The first character (`-`) identifies it as a regular file. The next three characters (`rw-`) show that the user has read and write permissions. The following three characters (`rw-`) show that the group has read and write permissions. The final three characters (`rw-`) show that others also have read and write permissions.

I used the `ls -la` command to display detailed information about all files and directories in the current directory, including hidden files. The `-l` option displays the long-format information, including ownership and permissions, while the `-a` option displays all files, including hidden files.

Example permission string:

`-rw-rw-rw-`

This indicates that `project_k.txt` is a regular file. The first character (`-`) identifies it as a regular file. The next three characters (`rw-`) show that the user has read and write permissions. The following three characters (`rw-`) show that the group has read and write permissions. The final three characters (`rw-`) show that others also have read and write permissions.

I used the `ls -la` command to display detailed information about all files and directories in the current directory, including hidden files. The `-l` option displays the long-format information, including ownership and permissions, while the `-a` option displays all files, including hidden files.

Example permission string:

`-rw-rw-rw-`

This indicates that `project_k.txt` is a regular file. The first character (`-`) identifies it as a regular file. The next three characters (`rw-`) show that the user has read and write permissions. The following three characters (`rw-`) show that the group has read and write permissions. The final three characters (`rw-`) show that others also have read and write permissions.

```
researcher2@f4a9d819e790:~$ pwd
/home/researcher2
researcher2@f4a9d819e790:~$ ls
projects
researcher2@f4a9d819e790:~$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Aug 24 06:25 .
drwxr-xr-x 1 root        root        4096 Aug 24 06:18 ..
-rw----- 1 researcher2 research_team  14 Aug 24 06:26 .bash_history
-rw-r--r-- 1 researcher2 research_team  220 Mar 27  2022 .bash_logout
-rw-r--r-- 1 researcher2 research_team 3574 Aug 24 06:18 .bashrc
-rw-r--r-- 1 researcher2 research_team 3574 Aug 24 06:18 .profile
drwxr-xr-x 3 researcher2 research_team 4096 Aug 24 06:18 projects
researcher2@f4a9d819e790:~$
```

Describe the permissions string

You can use `project_k.txt` as your example because its permissions clearly demonstrate all three permission groups.

The string is:

`-rw-rw-rw-`

Breakdown:

`- | rw- | rw- | rw-`
| user | group | others

Change file permissions

The file `project_k.txt` had write permission for others, which violated the organization's requirement that others must not have write access to files. I used the following command to remove write permission from others:

```
chmod o-w project_k.txt
```

The `o` represents others, `-w` removes write permission, and `chmod` modifies the file permissions. After running the command, `project_k.txt` had the permissions `-rw-rw-r--`, meaning the user and group can read and write to the file, while others can only read it.

```
drwx--x--- 2 researcher2 research_team 4096 Aug 24 05:47 draft
-rw-rw-rw- 1 researcher2 research_team  46 Aug 24 05:47 proje
.txt
-rw-r----- 1 researcher2 research_team  46 Aug 24 05:47 proje
.txt
-rw-rw-r-- 1 researcher2 research_team  46 Aug 24 05:47 proje
.txt
-rw-rw-r-- 1 researcher2 research_team  46 Aug 24 05:47 proje
.txt
researcher2@fad66adbe9c1:~/projects$ chmod o-w project_k.txt
researcher2@fad66adbe9c1:~/projects$
```

Change file permissions on a hidden file

The hidden file `.project_x.txt` is an archived file and should not have write permission for anyone. The user and group should have read access, while others should have no access. I used the following command:

```
chmod 440 .project_x.txt
```

The numeric permission `440` gives the user read permission (`4`), the group read permission (`4`), and others no permissions (`0`). The resulting permission string is `-r--r-----`.

```
.txt
researcher2@fad66adbe9c1:~/projects$ chmod u-w,g-w,g+r .project_x.
.txt
```

Change directory permissions

The `drafts` directory belongs to the `researcher2` user. Its current permissions allow the group to execute the directory, but the requirement is that only `researcher2` should be able to access the directory and its contents. I used the following command:

```
chmod 700 drafts
```

The `7` gives the owner read, write, and execute permissions, while the two `0` values give the group and others no permissions. The resulting permission string is `drwx-----`, which restricts access to the `researcher2` user.

Summary

In this activity, I used `ls -la` to examine file and directory permissions, including the hidden `.project_x.txt` file. I used `chmod` to remove unauthorized write permission from `project_k.txt`, give the appropriate read-only permissions to `.project_x.txt`, and restrict the `drafts` directory to the owner. These tasks demonstrate how Linux file permissions support authorization and help protect organizational information from unauthorized access.